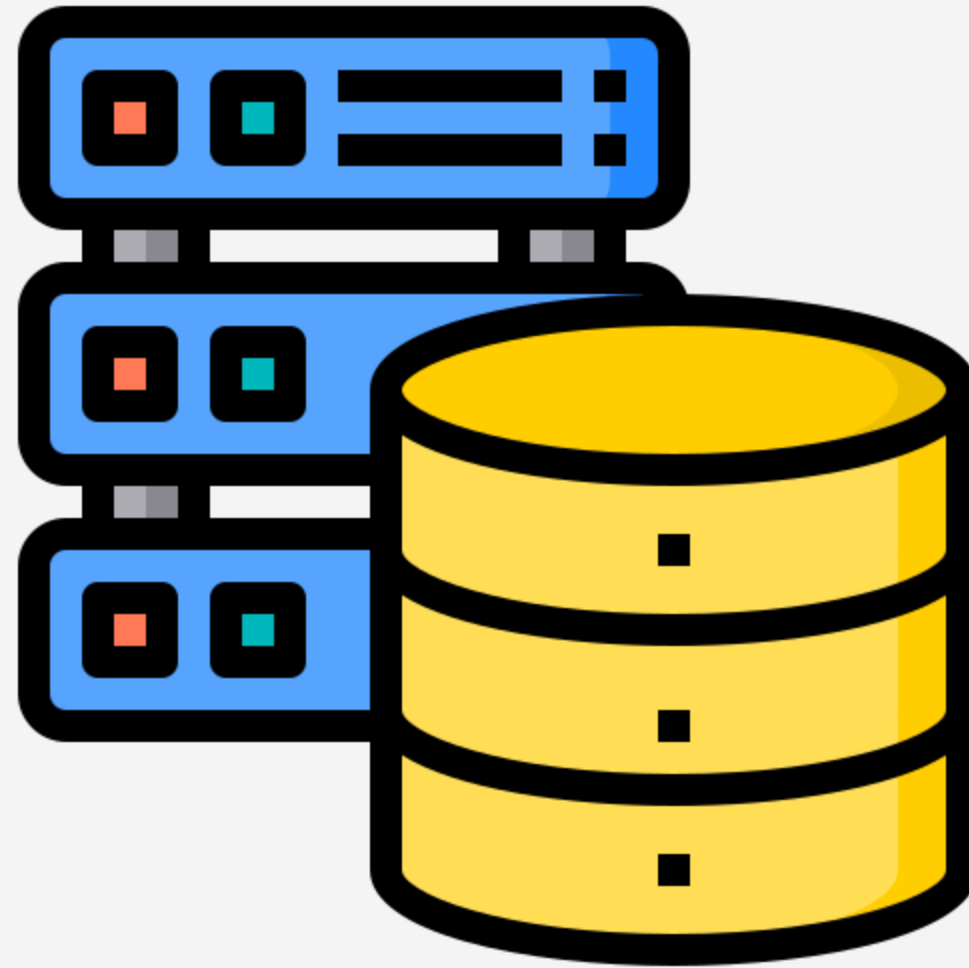


Yönetim Bilişim Sistemlerine Giriş

9. Hafta

Dr. Öğr. Üyesi Murat Kılınç

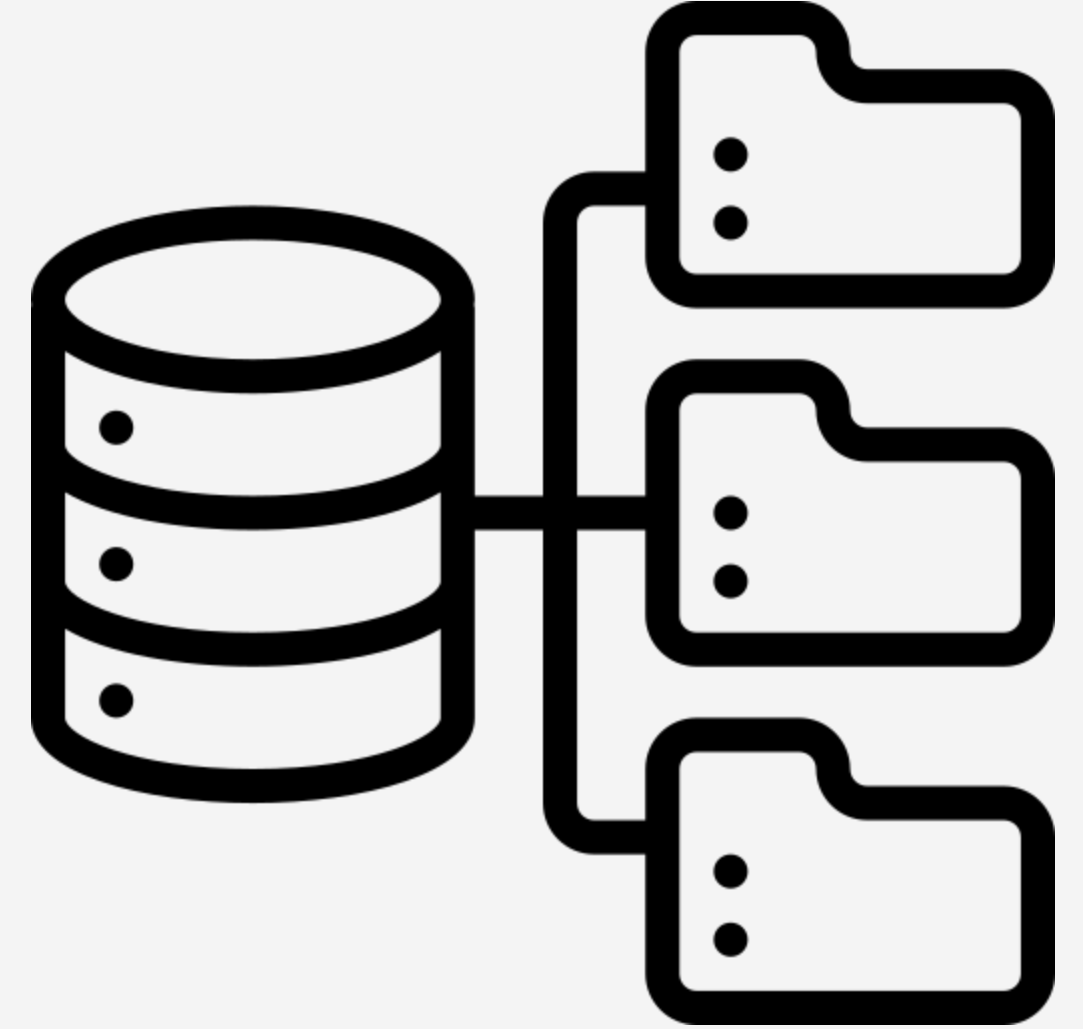
Veritabanı



Veritabanı Türleri

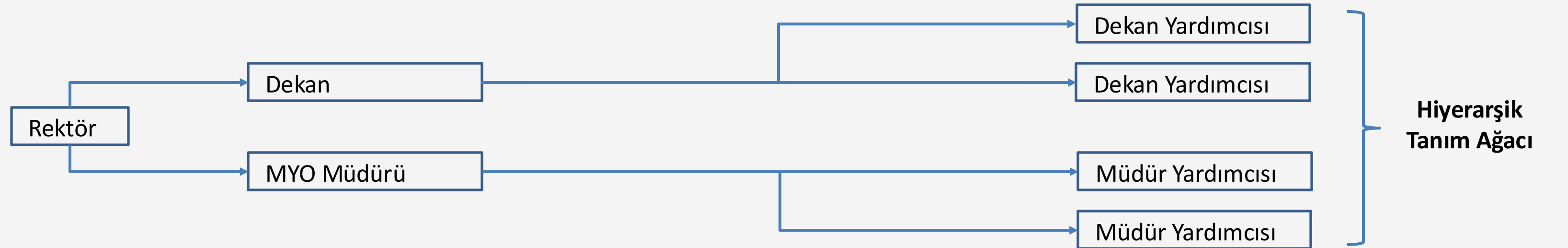
Veritabanı türleri, verileri yapılandırma, saklama ve bu verilere ulaşma biçimlerine göre farklılık gösterirler.

Günümüzde yaygın bir şekilde **ilişkisel veritabanları** kullanılmakla birlikte, **hiyerarşik** veritabanları ve **ağ veritabanları** zaman içinde kullanılmıştır. Son dönemlerde **nesne** tabanlı veritabanları da popüler olmaya başlamıştır.



Hiyerarşik Model

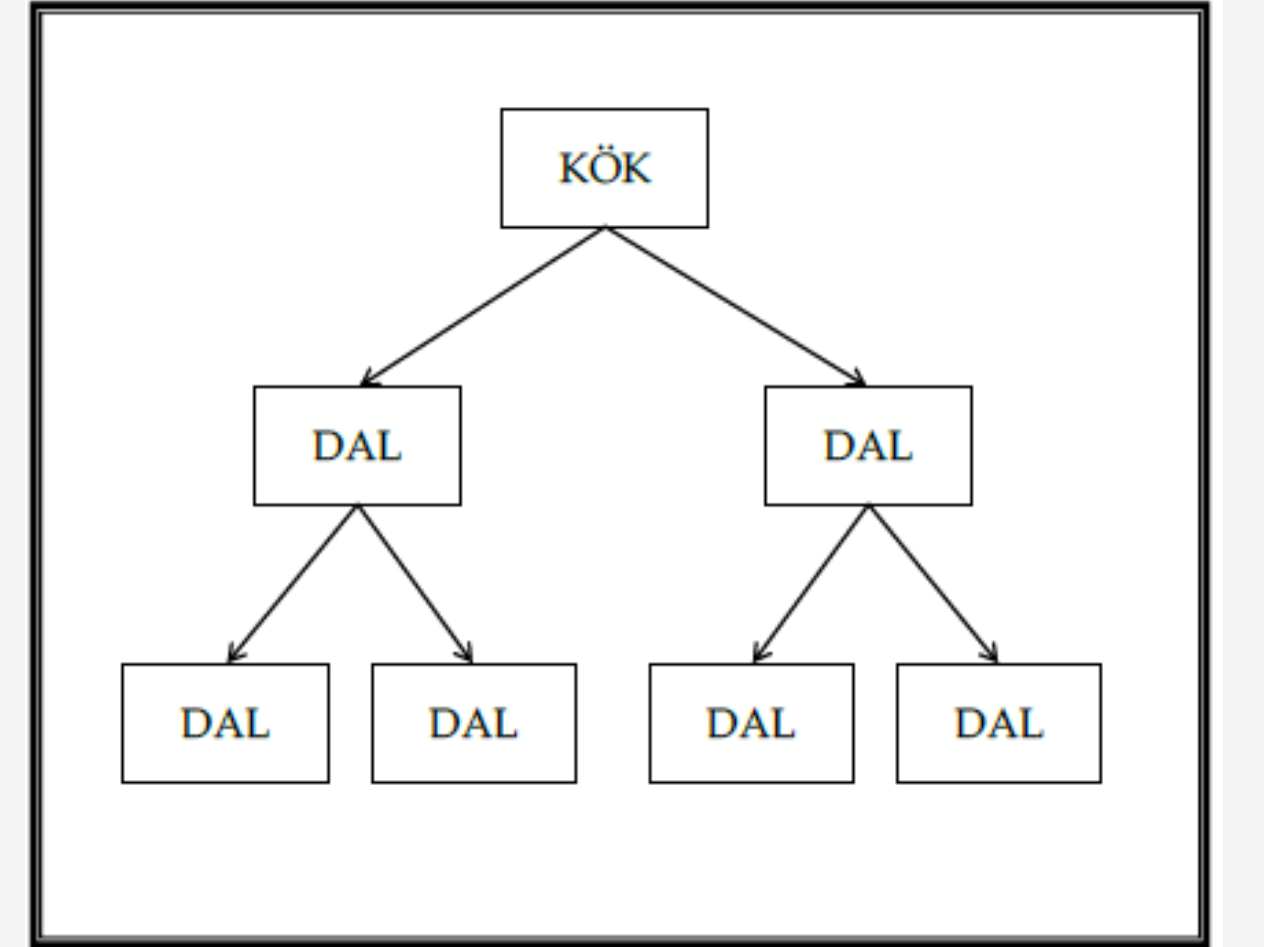
- Çoklu ilişkileri temsil edebilmek için, varlık tiplerinin gereksiz veri tekrarı yapmadan her ilişki için ayrı ayrı tanımlanmasına **Hiyerarşik veri modeli** denilir. Bu model, bir ağaç yapısına benzer.
- Model içerisindeki herhangi bir düğüm, altındaki n sayıda düğüme bağlanırken, kendisinin üstünde ancak bir düğüme bağlanabilir. Hiyerarşik yapının en tepesindeki düğüm noktasına kök denir ve bu düğümün sadece bağımlı düğümleri bulunur. Bu veri yapısını gösteren grafiğe de **hiyerarşik tanım ağacı** denir.



Hiyerarşik Model

- Her bir kayıt yalnızca bir ataya (parent) sahiptir ama sınırsız sayıda çocuğa (child) sahip olabilir.
- En üstte yer alan kayıt kök, altında yer alan kayıtlar ise dal olarak adlandırılır.
- Bu yapılarda, belirli bir veriye ulaşmak için tek bir yol bulunmaktadır.

Hiyerarşik yapıların **avantajları**, arama ve sıralama işlemlerinin çok hızlı olması ve çok geniş veri dosyalarını yönetebilmesidir. **Dezavantajları** ise veritabanının esnek olmaması, yalnızca **bire-çoklu** ilişkilere destek verebilmesi ve gereksiz verileri yapısında barındırması olarak ifade edilebilir.

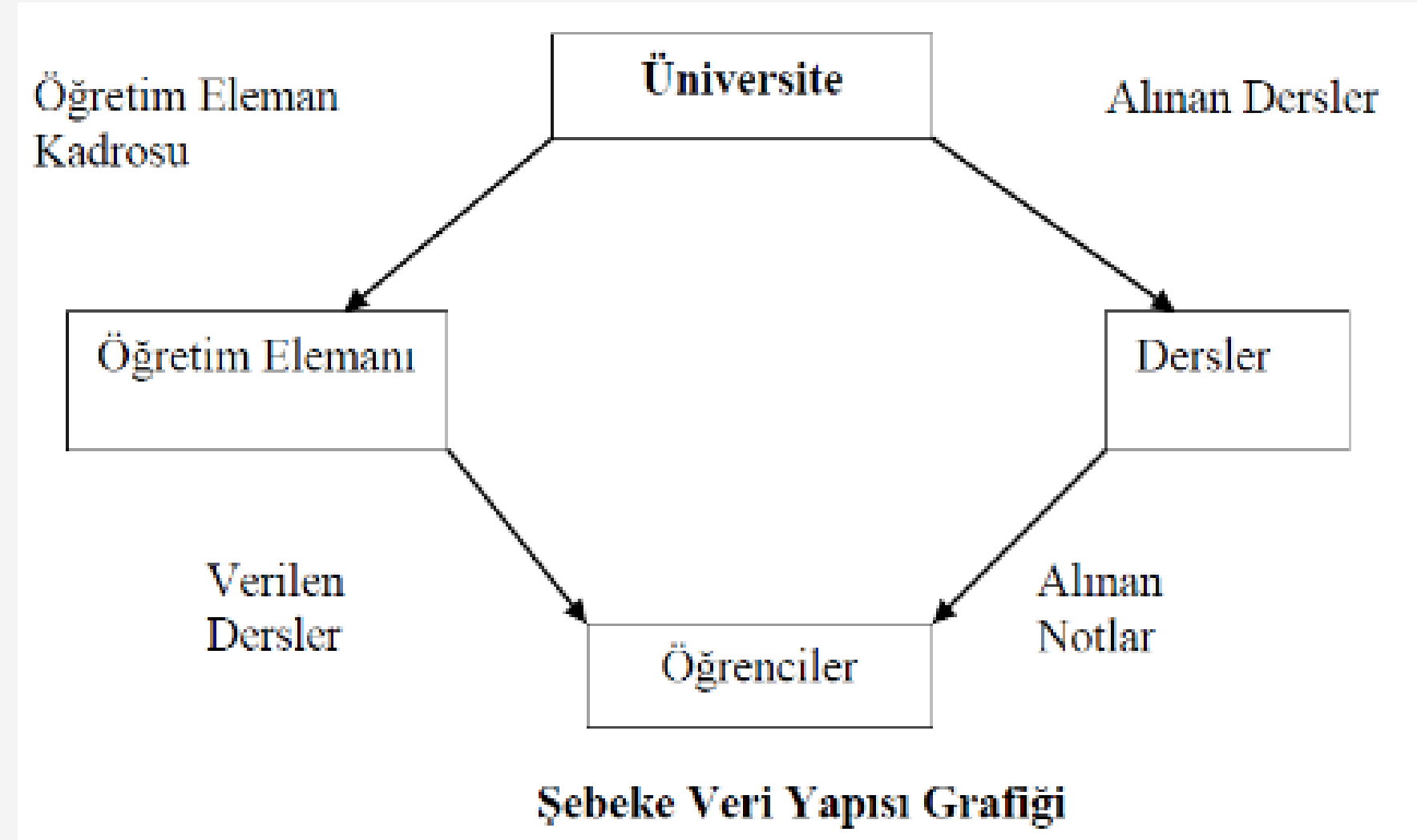


Şekil 5 : Hiyerarşik Veritabanı Yapısı

Ağ (Şebeke) Veri Modeli

Kayıt tipi ve bağlantı olmak üzere iki ayrı veri yapılandırma aracı vardır. Kayıt tipleri varlık tiplerini, bağlantılar ise ilişki tiplerini belirler. Bu yapıyı gösteren grafiğe de **şebeke veri yapısı grafiği** adı verilir. Veriler **düğüm (node)** ve **bağlantılar (edge)** şeklinde organize edilir.

Şebeke içinde bir eleman, herhangi başka bir elemana bağlanabilir. Hiyerarşik yapılardan farklı olarak, şebeke yapılarında bağlantı açısından herhangi bir sınırlama yoktur. Şebeke veri modeli, hiyerarşik modellere göre **daha az kısıtlayıcıdır ve daha karmaşık ilişkileri** temsil edebilir.



İlişkisel Veri Modeli

İlişkiler ve onların temsilleri olan tablolardan oluşan ilişkisel veri modeli ilk olarak 1970 yılında ortaya atılmıştır. İlişkisel veri modellerinde kullanılan tek yapılandırma aracı ilişkidir.

Sonraki örneklerde büyük harflerle yazılan ifadeler tablo isimlerini, parantez içindeki ifadeler de öznitelik isimlerini göstermektedir.

İlişkisel Veri Modeli

UNIVERSITE (UniversiteKodu, UniversiteAdi, Adres)

OGRETIMELEMANI (OgrKodu, OgrAdi, OgrSoyadi, Bolum, Brans)

OGRENCI (OgrenciNo, OgrenciAdi, OgrenciSoyadi, Bolum, Adres)

DERSLER (DersKodu, DersinAdi, Kredisi, Yariyil)

OGRENCINOTLAR (OgrenciNo, DersKodu, Vize, Final)

İlgili örnek basit bir üniversite veritabanının ilişkisel şemasını gösterilmektedir. İlişkisel şema, ilişki isimlerinin ve karşılık gelen tanım kümesi isimlerinin listesidir. Varlık tiplerini belirtmekte kullanılır.

İlişkisel şema listesini oluşturan **her bir satır, bir tablo olarak temsil edilir.** Tablonun sütunları öznitelik olarak isimlendirilir. Örneğin, Üniversite tablosunun öznitelikleri; **UniversiteKodu**, **UniversiteAdi** ve **Adres** bilgisidir.

İlişkisel Veri Modeli

Tabloda veri tekrarı olmaması için her satır diğerlerinden farklıdır ve tabloda bütün özniteliklerin aynı değerleri aldığı ikinci bir satır bulunmaz. **İlişki için bir anahtar kullanılması gerekir.** Anahtar, bir satırı tek başına tanımlayabilen öznitelikler kümesidir. Anahtar kavramı, ilişkisel veri modelinde kullanılan önemli bir kısıttır.

Universite

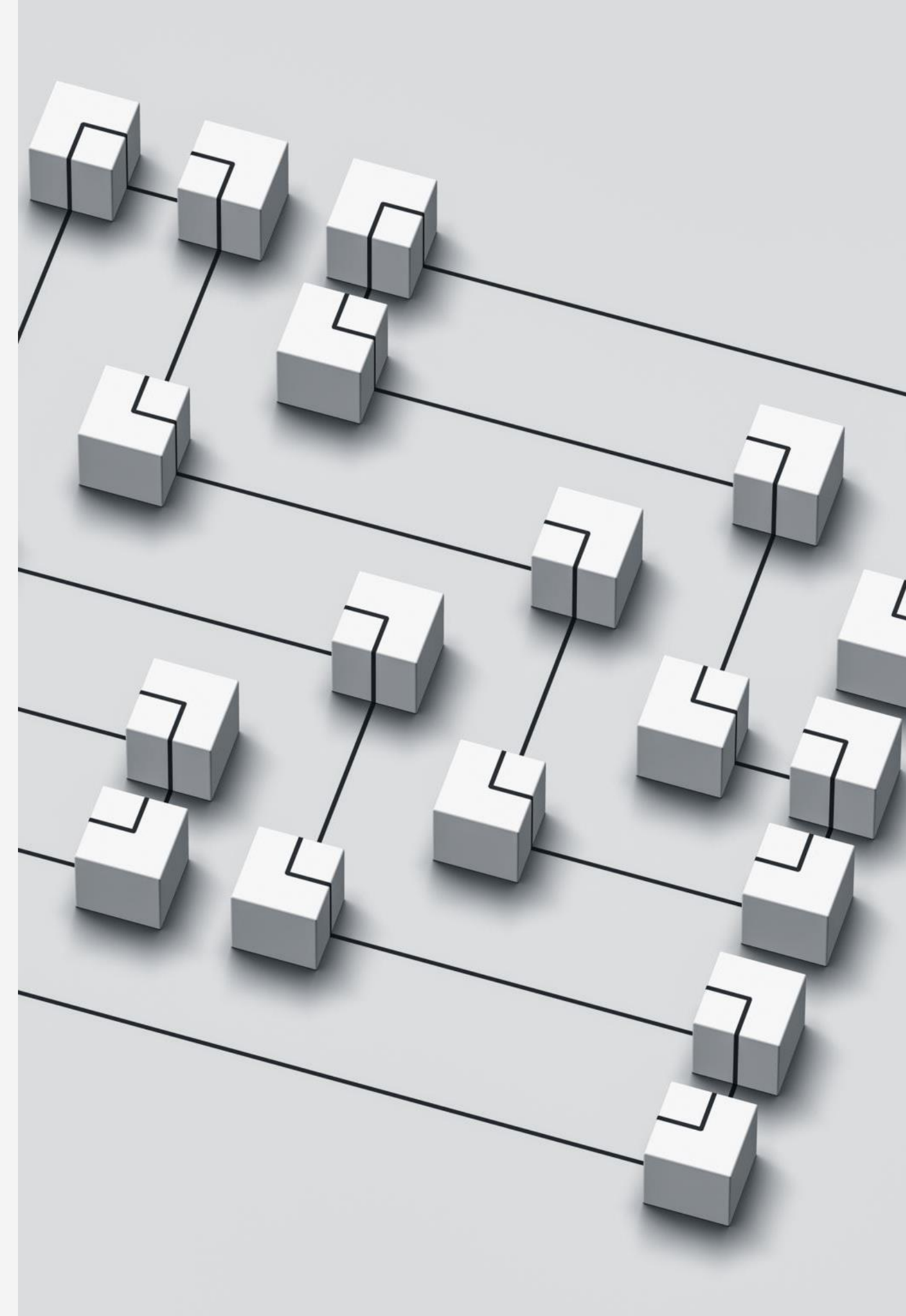
UniversiteKodu	UniversiteAdi	Adres
3300	Mersin Üniversitesi	Mersin
0600	ODTU	Ankara
3400	Yıldız Üniversitesi	İstanbul

Hatırlatma

➤ ilişkisel veritabanları da hem üreticileri, hem kullanım amaçları hem de çalıştıkları platform açısından farklılık göstermektedir.

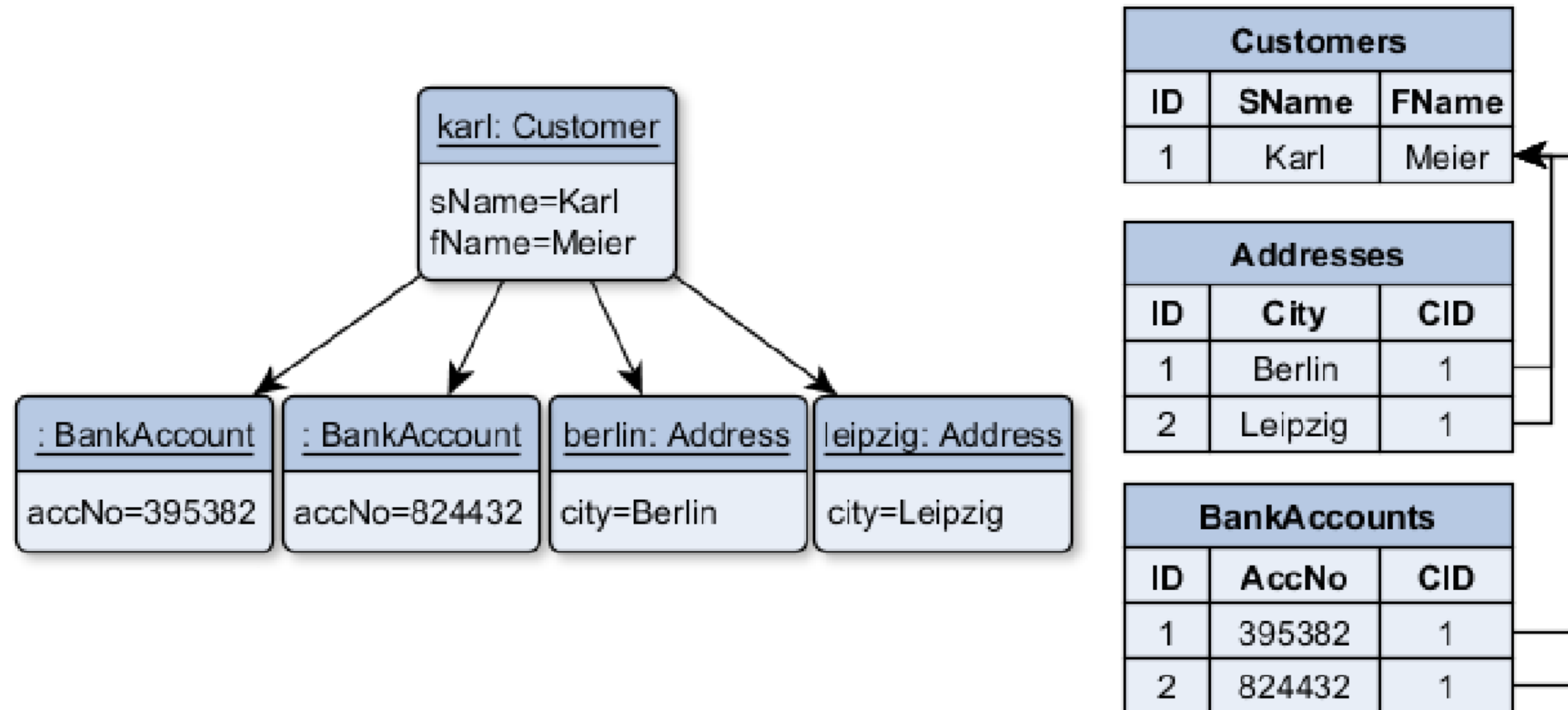
Örneğin;

- küçük boyutlu ve küçük maliyetli çözümler için **Access**,
- web uygulamaları için **MySQL**
- daha kapsamlı çözümler için daha yüksek maliyeti olan **Progress**, **MSSQL** ya da Linux üzerinde **PostgreSQL** akla gelmektedir.
- **Oracle** gibi çok yüksek maliyetli çözümler ancak veritabanını çok büyük olduğu ve üzerinde çok fazla işlem yapıldığı durumlarda düşünülmelidir.



Nesne Yönelimli Veri Modeli

Nesne Yönelimli Veri Modeli, verileri ve bu verilere yapılabilecek işlemleri bir arada tutan bir modeldir. Gerçek dünyadaki varlıkları ve onların özelliklerini modellemek için tasarlanmıştır. Bu model, programlama dillerinde kullanılan nesne yönelimli yaklaşımı veri tabanlarına taşır.



Nesne Yönelimli Veri Modeli

Nesneler (Object):

- Gerçek dünyadaki varlıklar nesne olarak tanımlanır.
- Örneğin, "Araba" bir nesnedir. Arabaya ait renk, model gibi özellikler (veriler) ve hızlanma, fren yapma gibi işlemler (metotlar) nesnenin içindedir.

Sınıflar (Class):

- Nesneler bir sınıfa aittir. Sınıf, benzer nesneleri gruplar.
- Örneğin, "Araba" bir sınıftır ve bu sınıfa ait birçok araba nesnesi olabilir (Ford, Toyota, vb.).

Kalıtım (Inheritance):

- Bir sınıf, başka bir sınıfın özelliklerini devralabilir.
- Örneğin, "Elektrikli Araba" sınıfı, "Araba" sınıfının özelliklerini (renk, model) alabilir ve üzerine yeni özellikler (pil kapasitesi) ekleyebilir.

Kimlik (Object ID):

- Her nesnenin kendine özgü bir kimliği vardır. Bu kimlik, nesneyi diğerlerinden ayırır.

Kapsülleme (Encapsulation):

- Veriler ve işlemler (metotlar), nesnenin içinde saklanır. Verilere yalnızca metodlarla erişilebilir, bu da güvenliği artırır.

Nesne Yönelimli Veri Modeli (Örnek)

Bir **öğrenci** nesnesini düşünün.

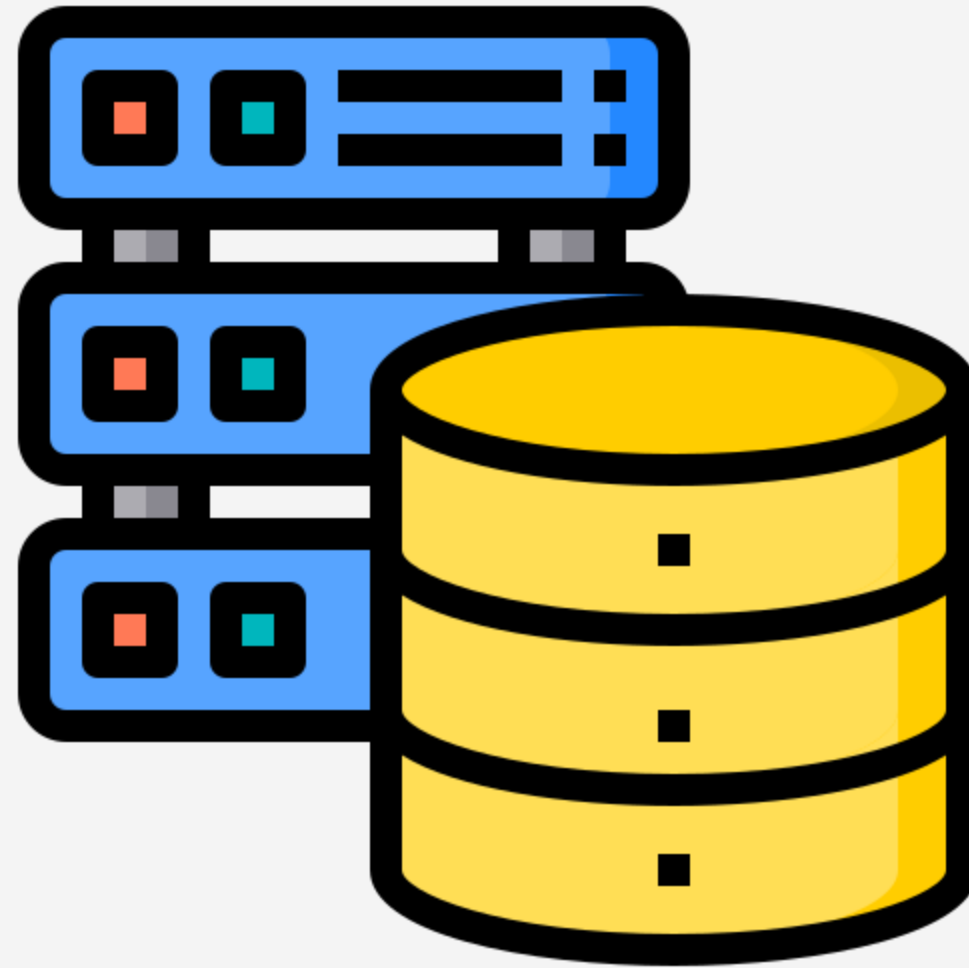
Özellikler (veriler): Adı, numarası, yaşı, sınıfı, bölümü, kayıt tarihi, not ortalaması (GPA), ders listesi, danışman öğretim üyesi.

Metotlar (işlemler): Ders kaydı yapma, not görüntüleme, devamsızlık takibi, ders bırakma, transkript oluşturma, profil güncelleme, kulüp üyeliği ekleme.

Bu modelde, öğrenciye ait bu veriler ve işlemler tek bir yerde (nesnede) tutulur. Bu sayede öğrenci nesnesine dair her şey düzenli bir şekilde organize edilmiş olur.



Varlık



Varlık Tanımı

- **Varlık (Entity):** Veritabanınızda bilgilerini tutmak istediğiniz şeyler, nesneler, yerler, olaylar veya kişilerdir.
- **Varlık Sınıfı:** Benzer karakteristiklere sahip **varlıkların bir araya gelerek** oluşturduğu bir kümedir. Bu kümelere örnek olarak **Müşteriler**, **öğrenciler**, **hastalar** verilebilir. Varlık sınıfı yerine bazen varlık kümeleri veya varlık tipleri gibi ifadelerde kullanılmaktadır.
- **Öznitelik:** Bir öznitelik (attribute) kayıt için belirlenen bir varlık özelliğidir. (Müşterinin yaşı, ilgili alanı, doğum tarihi vs.)



Varlık Tanımı

- Örneğin **Öğrenciler** isimli bir varlık sınıfını göz önüne alalım. Her bir varlık sınıfı için 1-5-10 veya daha fazla öznitelik tanımlanabilir.
- Bir varlık sınıfındaki her bir varlık, kendisini sınıftaki diğer varlıklardan ayıran en az bir özniteliğe sahip olmalıdır.
- Tek olan bir varlık, **ID (Identifier)** olarak çağrılır. Bu o varlığa ait tek olan bir kimlik bilgisi veya numarasıdır. Örneğin, bir işçiye ait sosyal güvenlik numarası veya bir öğrenciye ait okul numarası taktır.

Öğrenciler → Varlık Sınıfı

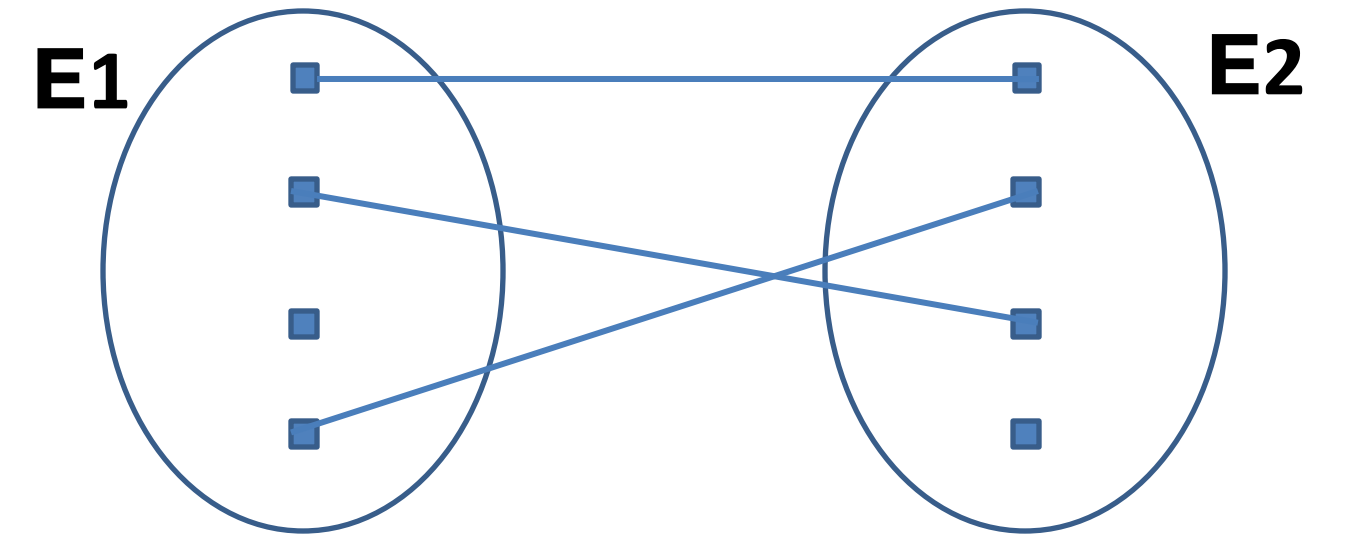
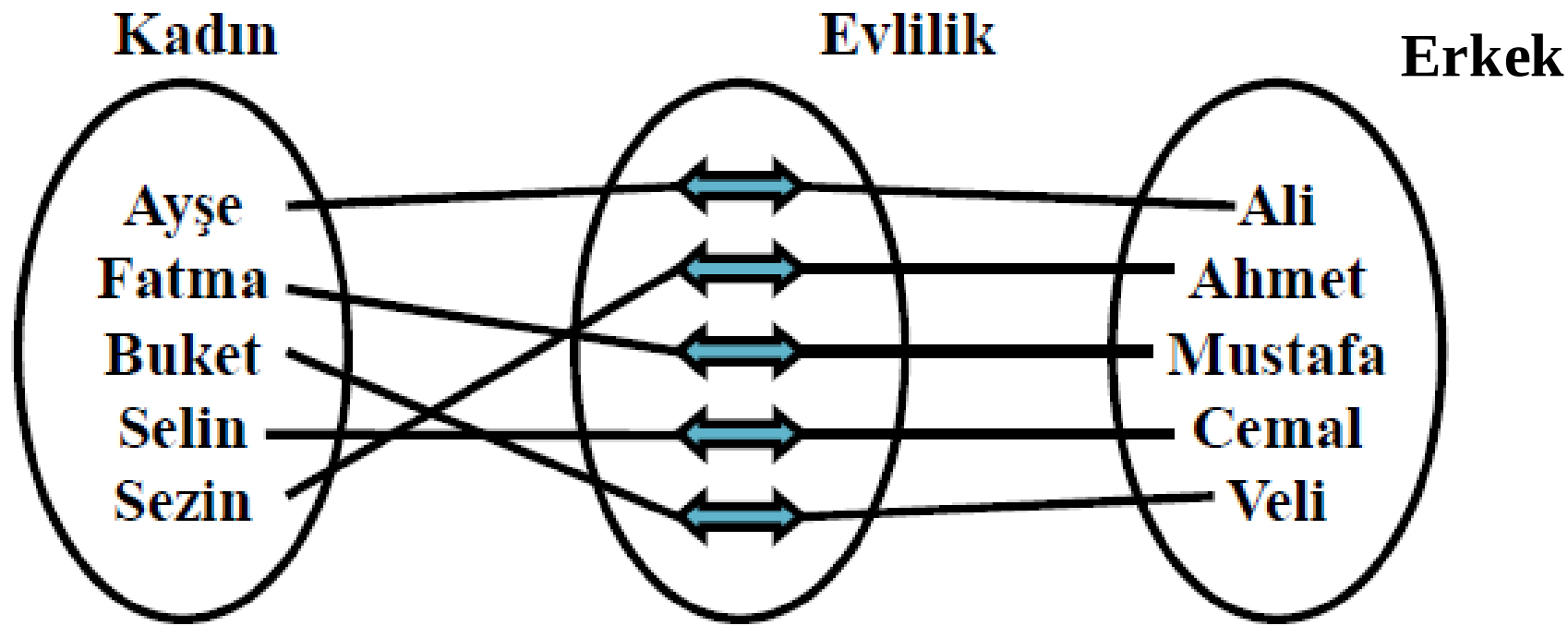
Öğrenci No	160316035
Adı	Aziz Yılmaz
Adresi	Güzelyurt Mah. 1234 sok.
Telefon	0555 444 33 22
Doğum Yeri	Manisa
Bölümü	Bilgisayar Programcılığı

Varlık

Bağıntı Türleri (1-1)

Birebir ilişkiler (one to one relationships): Aralarında bir ilişki olan iki tablo arasında, tablolardan birindeki asıl anahtar alanın kayıt değerinin, diğer tablodaki sadece bir kayıta karşılığının olması durumunu gösteren ilişki tipi.

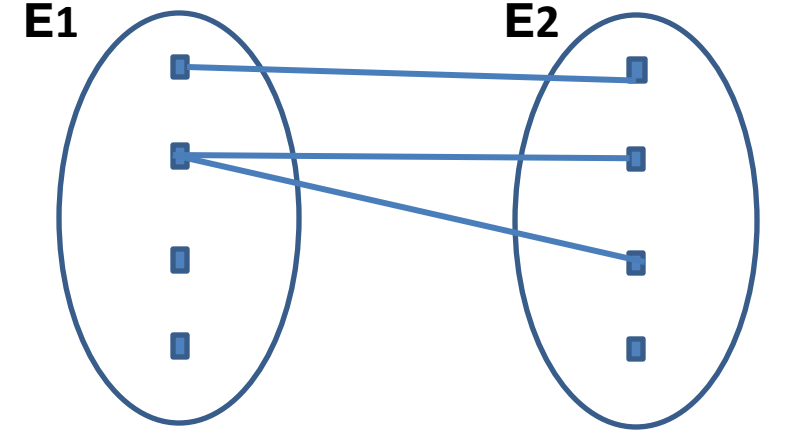
MEDENİ KANUN?



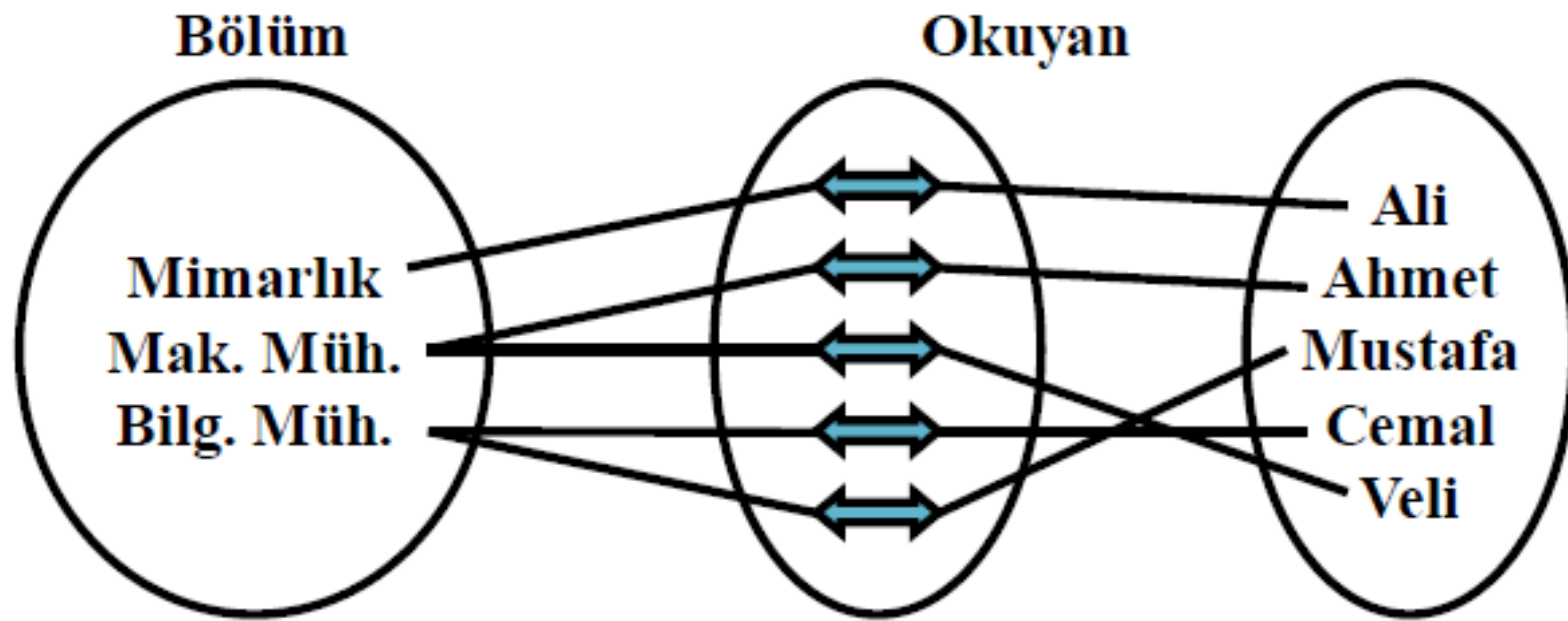
Bağıntı Türleri (1-N, N-1)

Tekil çoklu ilişkiler (one to many or many to one relationships):

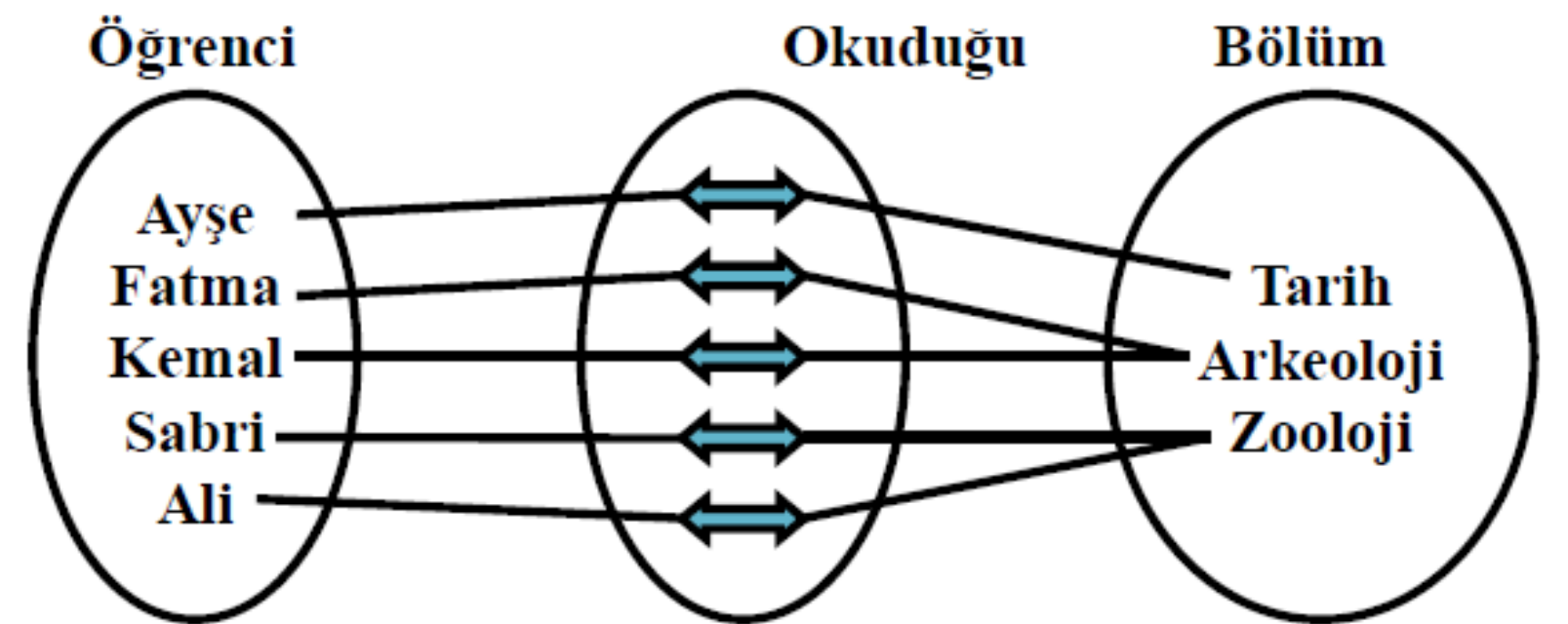
Aralarında bir ilişki olan iki tablo arasında, asıl anahtar alanın kayıt değerinin, diğer tablodaki birden fazla kayıta karşılığının olması durumunu gösteren ilişki tipi. **Örnek: Bir öğrencinin birden fazla almış olduğu derse ve bu derse ait vize, final ve sınav sonuçları gibi. Bir öğrenciye karşılık birden fazla ders notu.**



Birden-çoğa (one to many)

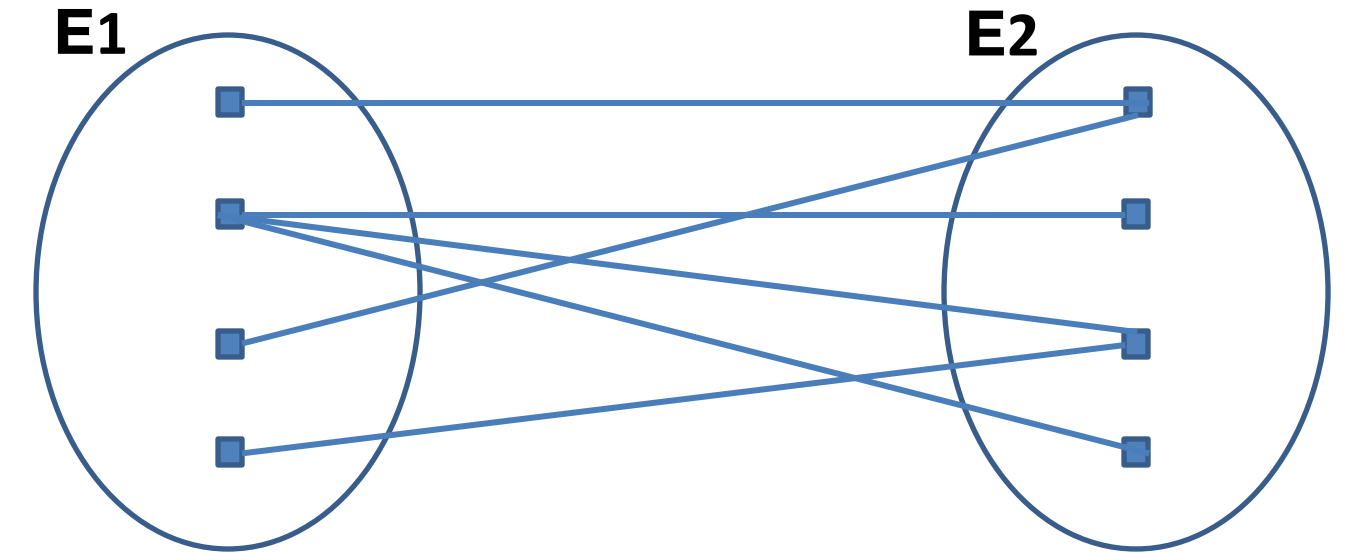
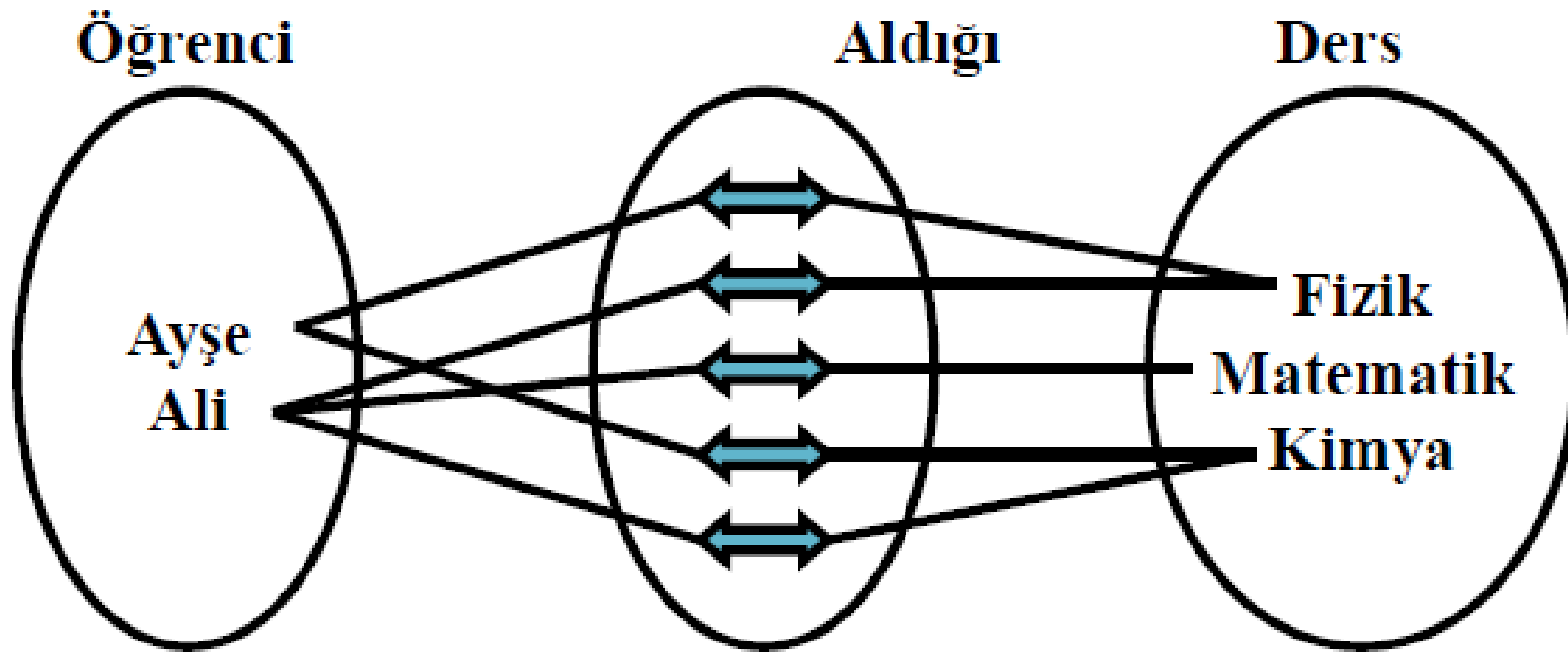


Çoktan-bire (many to one)



Bağıntı Türleri (N-N)

Çoklu ilişkiler (many to many relations): Aralarında bir ilişki olan iki tablo arasında, tablolardan herhangi birindeki herhangi bir kaydın, diğer tablodaki birden fazla kayıt ile ilişkilendirilebildiği ilişki tipi.





ALIŖTIRMA



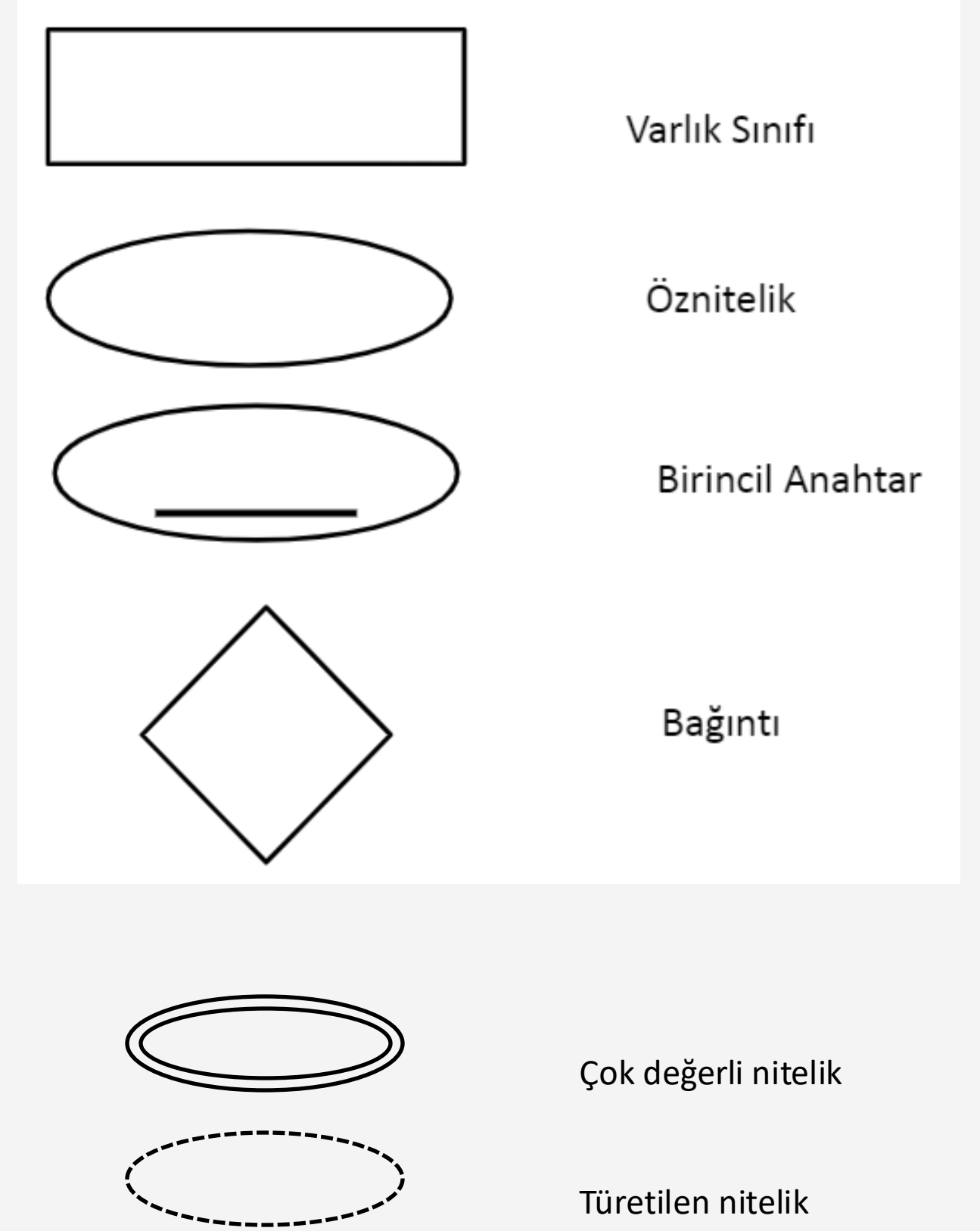
**Bu bağıntıları veritabanı için
nasıl ifade ediyoruz?**

Varlık İlişki Veri Modeli

Ticari veritabanlarında yaygın olarak kullanılan veri modellerinden biridir. Şebeke ve hiyerarşik veri modelleri ile ortak noktaları vardır; fakat, veritabanı tasarım süreçleri için kullanılmak maksadıyla geliştirildiklerinden bu iki modelin genelleştirilmiş şeklidir.

Çoklu ilişki tiplerinin doğrudan modelde kullanılmasına izin verir. Bu modelde, kurum şeması kavramı söz konusudur. Bu şema, kurumun tüm verisinin görünümünü temsil eder ve fiziksel sınırlamalardan bağımsızdır.

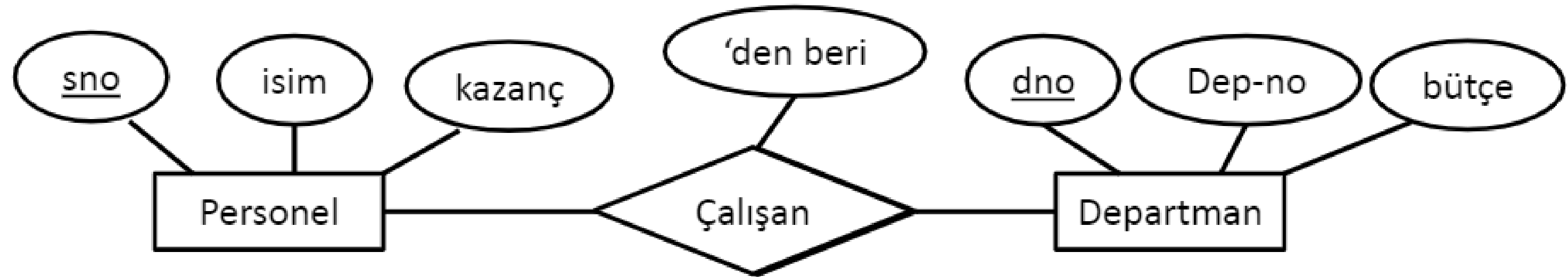
Temelde, Vİ veri modeli, veritabanının mantıksal özelliklerinin bir dokümantasyonudur. Vİ modeline göre düzenlenen veritabanının yapısı, **Varlık-İlişki Diyagramı** ile gösterilir. Şebeke ve hiyerarşik veri modellerinde, sadece ikili fonksiyonel bağlantılara izin verilmektedir.



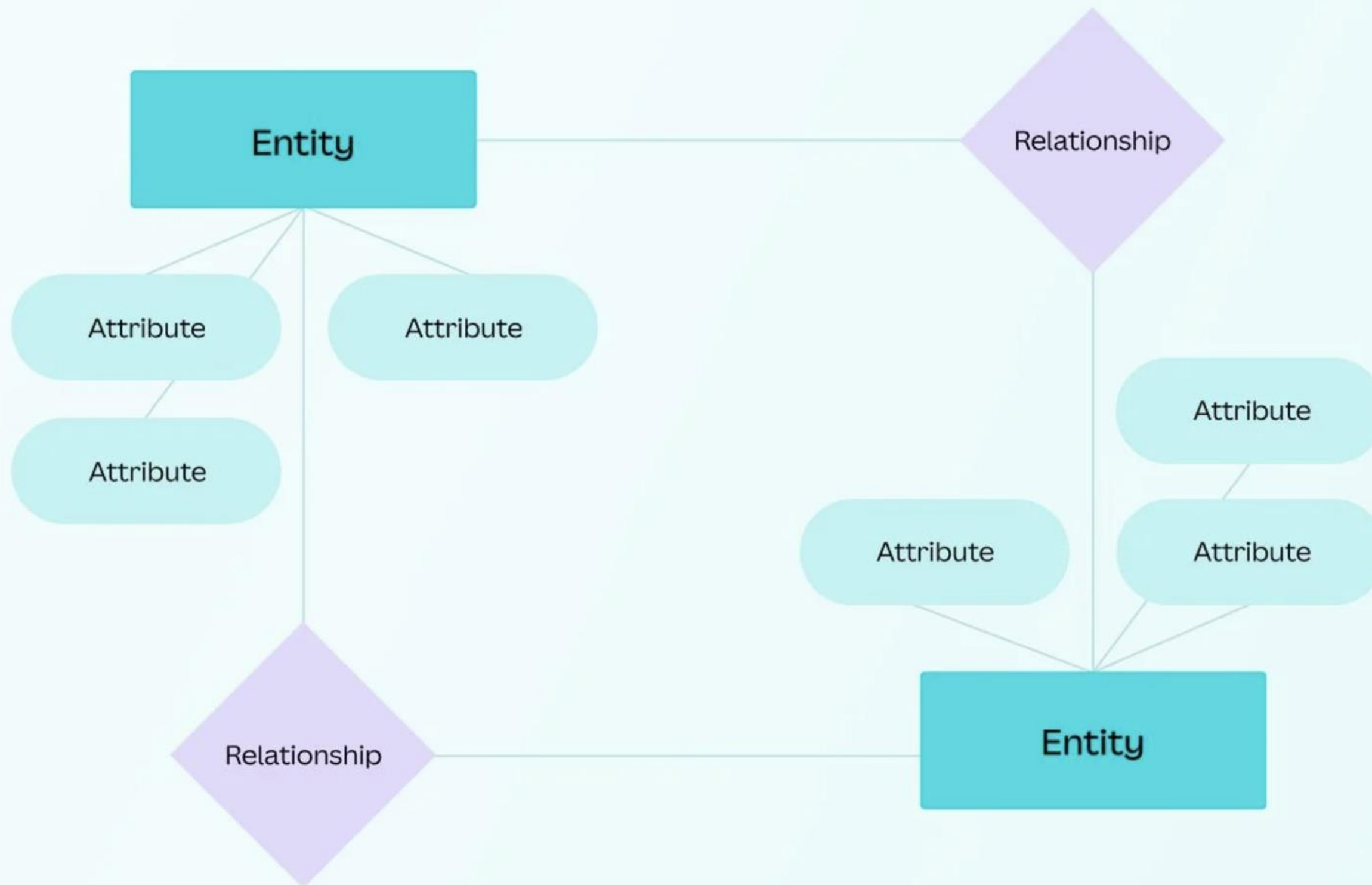
Varlık İlişki Veri Modeli

Vi veri modelinde varlıklar arasında **n adet** ilişki tanımlanabilir. Bu ilişkiler, birebir, fonksiyonel ve çoklu olabilir. Tekrar eden bağlantılarda kullanılabilir.

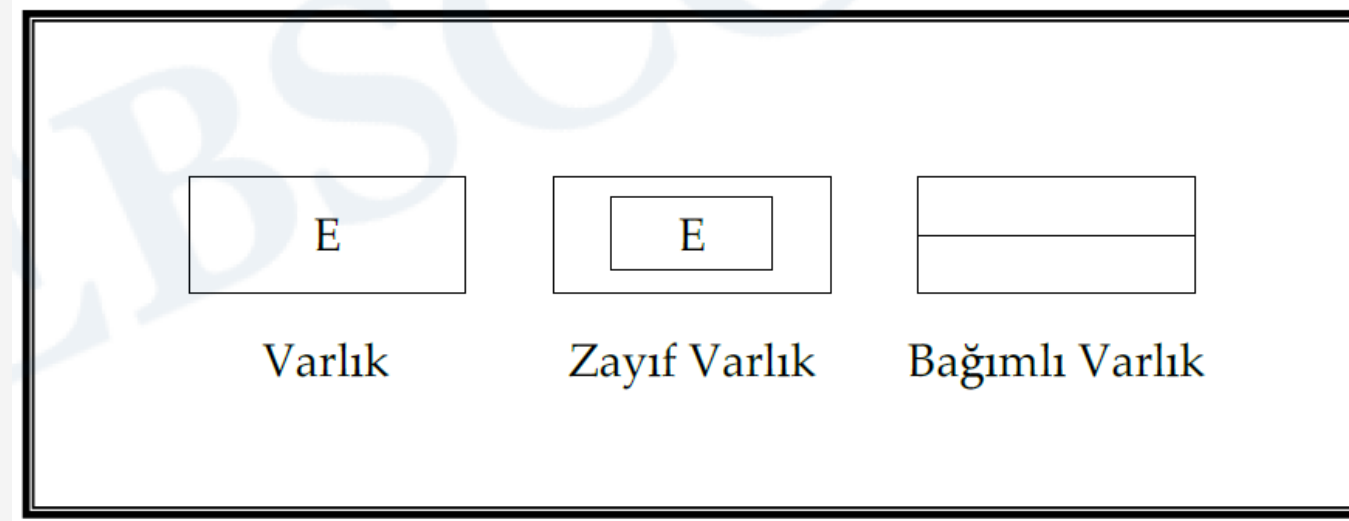
Vi modellerinde en büyük avantajlarından biri, **uzman olmayan kişiler tarafından da anlaşılabilir** yapıda olması ve üzerinde **düzeltilme** işlemlerinin kolayca yapılabilmesidir. Bu açıdan belirli bir veritabanı yönetim sistemine bağlı değildir.



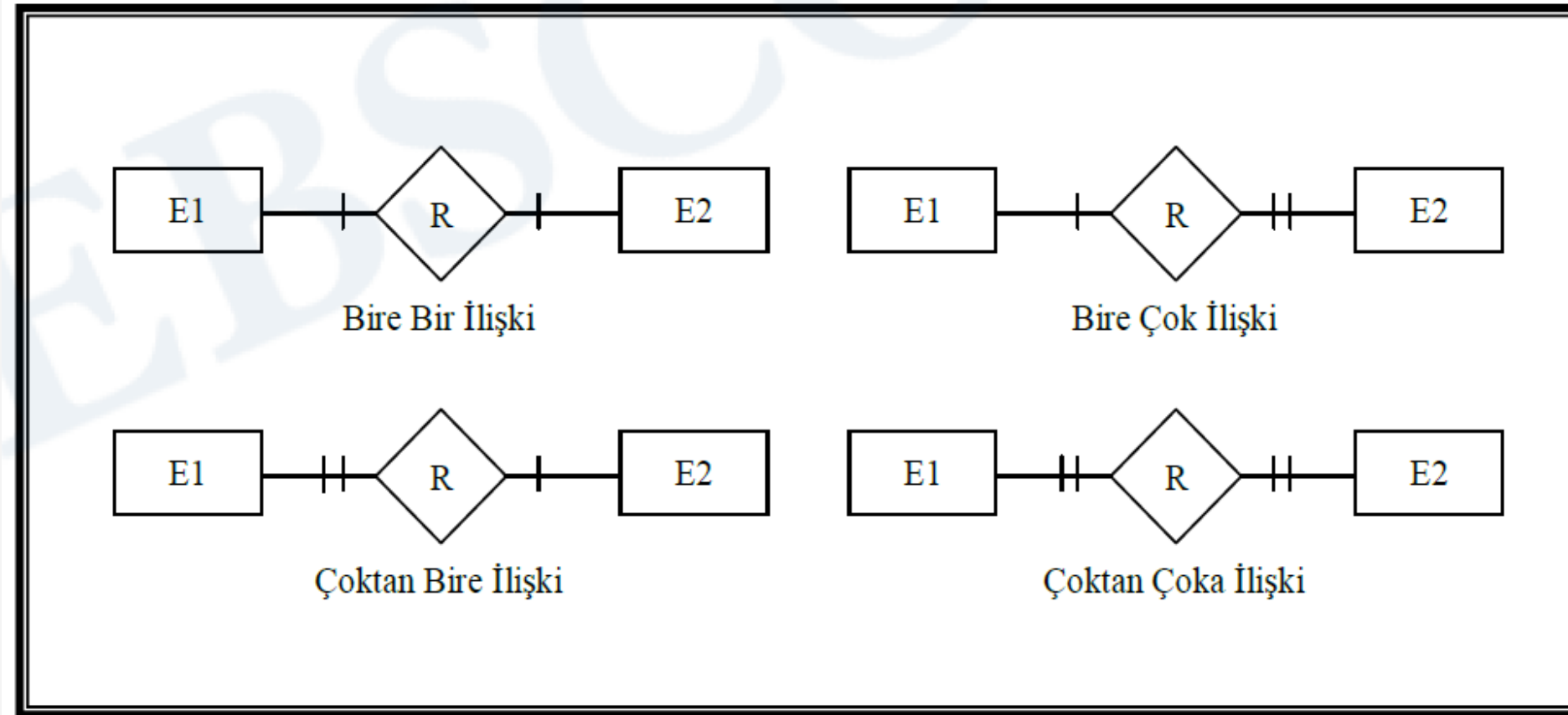
Varlık İlişki Veri Modeli



Varlık İlişki Şemaları



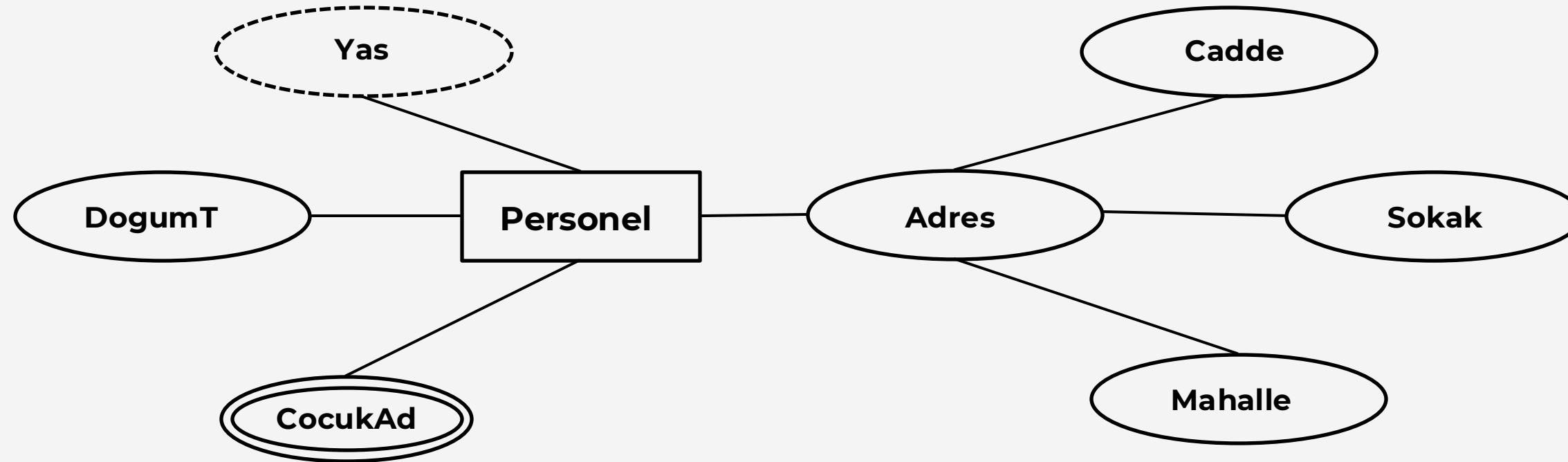
Şekil 9 : Varlıkları ifade eden Şemalar



Şekil 11 : İlişki Çeşitleri

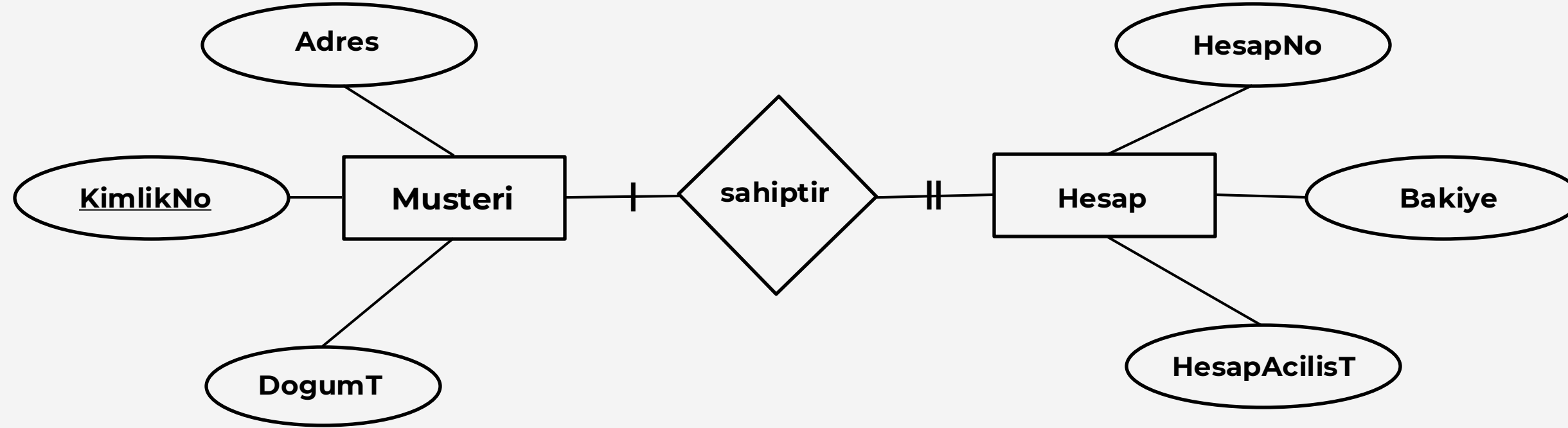
Örnek-1

Personel bilgilerini bir tablo olarak tasarlayıp E-R diyagramını çiziniz.



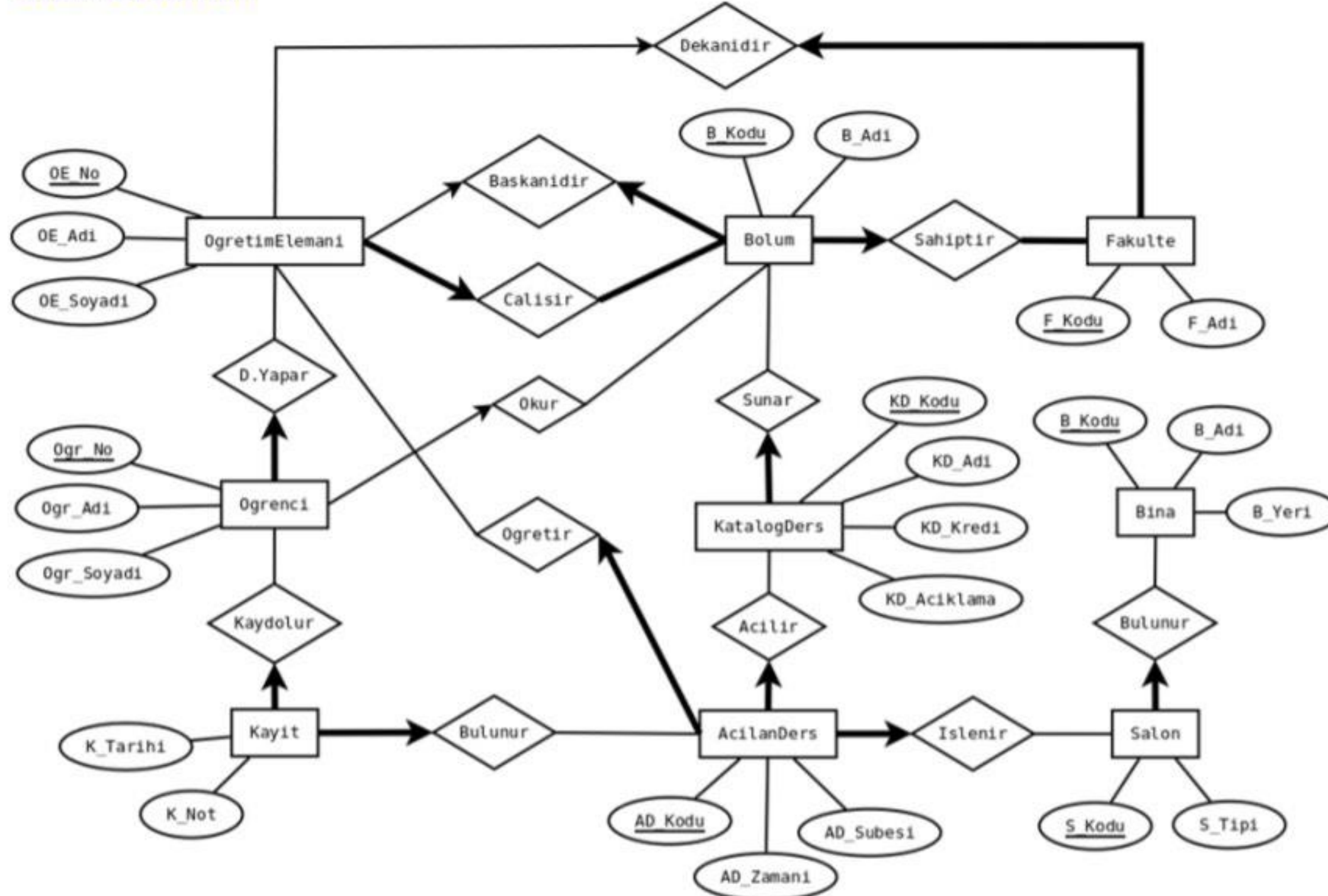
Örnek-2

Müşteri ve hesap bilgilerini içeren tabloları tasarlayıp E-R diyagramını çiziniz.



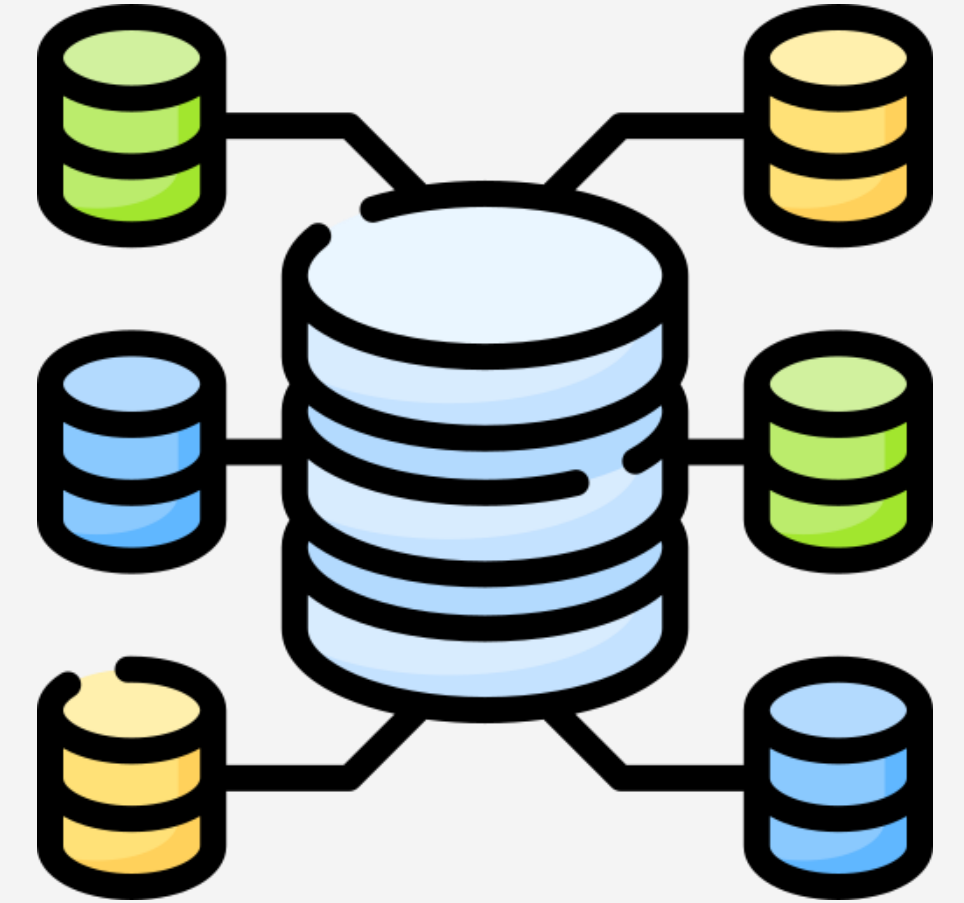
Karmaşık Yapılar

Chen Gösterimi



Veri Ambarı

- Veri ambarı, birbiri ile ilişkili verilerin sorgulanabildiği ve **analizlerinin** yapılabilirdiği bir depodur.
- İşletmelerde veri ambarları başlıca iki amaçla oluşturulmaktadır:
- Hareketsel ve işletmesel görevler arasındaki depo ve analitik stratejik verilerin birikimini sağlar. Bu veriler daha sonra yeniden kullanılmak üzere arşivlenir. Veri ambarları verilerin sorgulanabildiği ve analiz yapılabilirdiği bir depodur.
- Veri ambarlarının pazarda yeni fırsatlar bulmaya, rekabete katkı, yoğun proje çevirimi, iş, envanter, ürün maliyetlerinin azalmasını yanında farklı işlere ait verilerin ilişkilendirilmesi, **karar destek ve alınan bilgiye hızlı cevap verebilme** gibi birçok katkısı vardır.



Veri Ambarı İşlevleri

- Değişik platformlar üzerindeki işletimsel uygulamalara ait verilere erişim ve gerekli verilerin bu platformlardan alınması,
- Alınan verilerin temizlenmesi, tutarlı duruma getirilmesi, özetlenmesi, birleştirmesi ve birbiriyle bütünleştirilmelerinin sağlanması,
- Dönüştürülen verilerin veri ambarı ortamına dağıtılması,
- Gönderilen verilerin bir veritabanında toplanması,
- Depolanan bilgi ile tanımlanmış verilerin veri kataloğunda saklanması ve son kullanıcılara sunulması,
- Veri ambarında bulunan bilgileri son kullanıcıların karar destek amaçlı kullanımının sağlanması.

Veri Ambarı ve VT Farklar

Amaç:

- Veri Ambarı:** Analitik sorgular, raporlama ve iş zekası için kullanılır.
- Veritabanı:** Günlük operasyonel işlemleri yönetmek için kullanılır.

Veri Yapısı:

- Veri Ambarı:** Özetlenmiş, toplulaştırılmış ve tarihsel veri içerir (Boyutsal şemalar - Star veya Snowflake yapıları).
- Veritabanı:** Detaylı, ham ve güncel veri içerir (İlişkisel şemalar).

Performans:

- Veri Ambarı:** Karmaşık analitik sorgulara ve raporlamaya optimize edilmiştir.
- Veritabanı:** Hızlı okuma, yazma ve güncelleme işlemleri için optimize edilmiştir.

Güncellenme Sıklığı:

- Veri Ambarı:** Veriler toplu işlemlerle (batch processing) belirli aralıklarla güncellenir.
- Veritabanı:** Veriler sürekli olarak gerçek zamanlı güncellenir.

Kullanıcılar:

- Veri Ambarı:** İş analistleri, yöneticiler ve veri bilimciler tarafından kullanılır.
- Veritabanı:** Yazılımcılar, uygulamalar ve son kullanıcılar tarafından kullanılır.